Mississippi Valley State University

CS 341 — Discrete Structures

Lecture 46: Writing Down the Equation

Finding Recurrences

Lehman, Leighton & Meyer, *MCS* Chapter 21

Name: _____ Date: _____

A change of textbook again

Chartrand defines recursive sequences but does not solve them. These three lectures use the free supplement, *MCS* Chapter 21, which does.
Last lecture ended with merge sort's $T(n) = 2T(n/2) + n$ and the promise that we would learn to solve equations like it. Today we learn to *write* them; solving comes Wednesday.

By the end of today you should be able to

1. Set up a recurrence from a problem description.
2. Compute terms and conjecture a closed form.
3. Say why a conjecture from terms is not yet a proof.

1. What a recurrence is

Definition

A **recurrence** defines a function on the positive integers by giving one or more **boundary conditions** and an equation expressing $f(n)$ in terms of earlier values.

You met these in Lecture 30 as recursively defined sequences. The new question is how to produce one from a problem, and then how to get a closed form.

2. Towers of Hanoi

Three posts, n disks of distinct sizes stacked on the first, largest at the bottom. Move the whole stack to another post, one disk at a time, never placing a larger disk on a smaller one.

Let t_n be the fewest moves required.

Find the recurrence. To move the largest disk, the other $n-1$ must first be stacked on the spare post — that takes t_{n-1} moves. Then move the largest (1 move). Then move the $n-1$ back on top (t_{n-1} again).

$$t_0 = 0, \qquad t_n = 2t_{n-1} + 1 \quad (n \geq 1)$$

n	0	1	2	3	4	5
t_n	0	1	3	7	15	31

Conjecture: $t_n = 2^n - 1$.

The conjecture is not yet proved

Six values agree. Week 1 taught you that forty values are not enough, so six certainly are not. Two ways to close the gap: **prove it by induction**, which you can already do, or **solve the recurrence**, which is Wednesday. The second is better, because it also works when you cannot guess the answer.

The legend. With 64 disks at one move per second, the monks need $2^{64} - 1$ seconds — about 585 billion years. The 2^n from Day 1, with consequences.

3. Climbing stairs

A staircase of n steps may be climbed by taking single steps or double hops. How many ways?

Let $f(n)$ count them. A climb either begins with a step, leaving $n-1$ stairs, or with a hop, leaving $n-2$. The two cases are disjoint, so by the Addition Principle:

$$f(0) = 1, \quad f(1) = 1, \qquad f(n) = f(n-1) + f(n-2)$$

n	0	1	2	3	4	5	6
$f(n)$	1	1	2	3	5	8	13

That is the Fibonacci sequence.

The case split is Lecture 24

"The climb begins with a step *or* a hop" is an exhaustive, disjoint case split, and the two counts are *added* — Lecture 31. Setting up a recurrence is a counting argument with the answer written in terms of smaller instances of itself.

4. Merge sort

Sorting n items: split into two halves, sort each, merge. Let $T(n)$ count comparisons.

Two subproblems of size $n/2$, plus at most $n-1$ comparisons to merge (Theorem 6.43):

$$T(1) = 0, \qquad T(n) = 2T(n/2) + n - 1$$

n	1	2	4	8
$T(n)$	0	1	5	17

This one is a different shape

Hanoi reduces n to $n-1$. Merge sort reduces n to $n/2$ — it reaches *halfway back*, not one step back.
That difference is why the two need different solution methods, and why their answers are 2^n apart in cost. Wednesday handles the first kind; Friday the second.

Exit ticket

1. A sequence satisfies $a_1 = 2$ and $a_n = 3a_{n-1}$. Compute a_1 through a_4 and conjecture a formula.

2. Why is computing six terms not a proof of your conjecture?

Before next class

Read *MCS* Section 21.3, "Linear Recurrences," pp. 874–883. The link is on Canvas.

Questions I still have